

模型重用

Modelica语法详解

苏州同元软控信息技术有限公司版权所有，
未经授权不得转载、复制或传播或以其他方式使用，
侵权必究

苏州同元软控信息技术有限公司

2024年2月27日



TONGYUAN
Software and Control

目录

1. 继承重用

2. 实例化重用

3. 重声明重用

4. 本章回顾

1. 继承重用

仔细观察下面的三个模型的代码，有何特点？

```
model Resistor
import SI=Modelica.Siuints;
Parameter SI.Resistance R(start=1)=1;
SI.Voltage v;
SI.Current i;
...Interfaces.PositivePin p;
...Interfaces.NegativePin n;
equation
V=p.v-n.v;
0=p.i+n.i;
i=p.i;
R*i=v;
end Resistor
```

电阻

```
model Capacitor
import SI=Modelica.Siuints;
Parameter SI.Resistance C(start=1)=1;
SI.Voltage v;
SI.Current i;
...Interfaces.PositivePin p;
...Interfaces.NegativePin n;
equation
V=p.v-n.v;
0=p.i+n.i;
i=p.i;
i=C*der(v);
end Capacitor
```

电容

```
model Inductor
import SI=Modelica.Siuints;
Parameter SI.Resistance L=1;
SI.Voltage v;
SI.Current i;
...Interfaces.PositivePin p;
...Interfaces.NegativePin n;
equation
V=p.v-n.v;
0=p.i+n.i;
i=p.i;
L*der(i)=v;
end Inductor
```

电感

三个组件模型，绝大部分代码是相同的，出现大量的重复代码，给后期代码的修改和维护造成很大的不便。

冗余是一切罪恶的根源



1. 继承重用

再阅读下面的代码，和前面的代码进行比较，有何不同？

```
partial model PartialPort
  SI.Voltage v;
  SI.Current i;
  Interfaces.PositivePin p;
  Interfaces.NegativePin n;
equation
  v = p.v - n.v;
  0 = p.i + n.i;
  i = p.i;
end PartialPort;
```

基类

```
model Inductor
  parameter SI.Inductance L = 1;
  extends PartialPort;
equation
  L * der(i) = v;
end Inductor;
```

电阻

```
model Capacitor
  parameter SI.Capacitance C = 1;
  extends PartialPort;
equation
  i = C * der(v);
end Capacitor;
```

电容

```
model Resistor
  parameter SI.Resistance R = 1;
  extends PartialPort;
equation
  R * i = v;
end Resistor;
```

电感

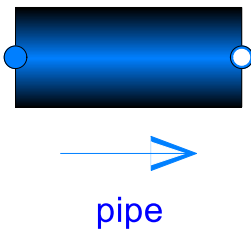
通过继承重用的方式，大大简化组件模型代码，避免不必要的冗余，避免了模型维护和修改时的繁琐。



1. 继承重用-使用场景

- 1. 继承connector, 设计不同的图标和连接线
- 2. 继承partial类, 完善模型行为方程

场景1：继承connector, 设计不同的图标和连接线



```
connector FluidPort
  replaceable package Medium =
    Modelica.Media.Interfaces.PartialMedium
  annotation (choicesAllMatching = true);
  flow Medium.MassFlowRate m_flow;
  Medium.AbsolutePressure p;
  stream Medium.SpecificEnthalpy h_outflow;
  stream Medium.MassFraction Xi_outflow[Medium.nXi];
  stream Medium.ExtraProperty C_outflow[Medium.nC];
end FluidPort;
```

原始接口

```
connector FluidPort_a "inlet"
  extends FluidPort;
  annotation (defaultComponentName = "port_a",
    Icon(coordinateSystem(preserveAspectRatio = false,
      extent = {{-100, -100}, {100, 100}}, graphics = {Ellipse(
        extent = {{-100, 100}, {100, -100}},
        lineColor = {0, 127, 255},
        fillColor = {0, 127, 255},
        fillPattern = FillPattern.Solid), Ellipse(
        extent = {{-100, 100}, {100, -100}},
        lineColor = {0, 0, 0},
        fillColor = {0, 127, 255},
        fillPattern = FillPattern.Solid)})));
end FluidPort_a;
```

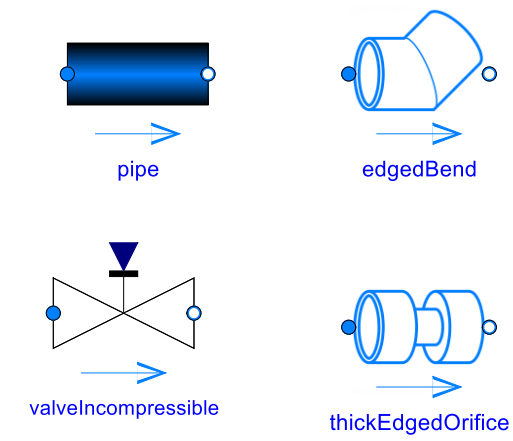
入口

```
connector FluidPort_b "outlet"
  extends FluidPort;
  annotation (defaultComponentName = "port_b",
    Icon(coordinateSystem(preserveAspectRatio = false,
      extent = {{-100, -100}, {100, 100}}, graphics = {
        Ellipse(
          extent = {{-100, 100}, {100, -100}},
          lineColor = {0, 127, 255},
          fillColor = {0, 127, 255},
          fillPattern = FillPattern.Solid),
        Ellipse(...),
        Ellipse(...)})));
end FluidPort_b;
```

出口

1. 继承重用-使用场景

场景2：继承partial类，完善模型行为方程



```
partial model PartialTwoPort "Partial component with two ports"
  import Modelica.Constants;
  outer Modelica.Fluid.System system ;
  replaceable package Medium =
    Modelica.Media.Interfaces.PartialMedium
    "Medium in the component"
    annotation (choicesAllMatching = true);
  parameter Boolean allowFlowReversal = system.allowFlowReversal
    "= true (port_a -> port_b)"
    annotation (Dialog(tab = "Assumptions"));
  Modelica.Fluid.Interfaces.FluidPort_a port_a(...)
    "Fluid connector a"
    annotation (...);
  Modelica.Fluid.Interfaces.FluidPort_b port_b(...)
    "Fluid connector b"
    annotation (...);
end PartialTwoPort;
```

```
model StaticPipe "pipe"
  extends PartialTwoPort;
  parameter Medium.AbsolutePressure p_a_start = system.p_start
    "Start value of pressure at port a"
  annotation (Dialog(tab = "Initialization"));
  FlowModel flowModel(...)
  equation
    .....
  annotation (defaultComponentName = "pipe", ...);
end StaticPipe;

model Valve "Valve"
  extends PartialTwoPort;
  constant SI.ReynoldsNumber Re_turbulent = 4000;
  parameter Boolean use_Re = system.use_eps_Re;
  SI.AbsolutePressure dp_turbulent = if not use_Re then dp_small
    max(dp_small, (Medium.dynamicViscosity(state_a) + ...);
  equation
    .....
end Valve;

model CurvedBend "CurvedBend"
  extends PartialTwoPort;
  constant SI.ReynoldsNumber Re_turbulent = 4000;
  parameter Medium.AbsolutePressure dp_small(min = 0) =
    PressureLoss.Bend.dp_curvedOverall_DP(...);
  equation
    .....
end CurvedBend;

model ThickEdgedOrifice "Thick edged orifice"
  extends PartialTwoPort;
  parameter Medium.AbsolutePressure dp_small(min = 0) =
    PressureLoss.Orifice.dp_thickEdgedOverall_DP(...);
  equation
    .....
end ThickEdgedOrifice;
```

每个模型都需要一个入口和一个出口，并且都需要选择流过模型的介质

1. 继承重用-一般结构

```
record ColorData
  parameter Real red = 0.2;
  parameter Real blue = 0.6;
  Real green;
end ColorData;
```

```
model Color
  extends ColorData;
equation
  red + blue + green = 1;
end Color;
```

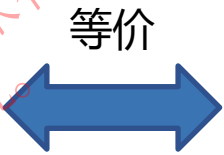


```
model Color
  parameter Real red = 0.2;
  parameter Real blue = 0.6;
  Real green;
equation
  red + blue + green = 1;
end Color;
```

继承时要改变值怎么办？

```
record ColorData
  parameter Real red = 0.2;
  parameter Real blue = 0.6;
  Real green;
end ColorData;
```

```
model Color
  extends ColorData(red = 0.3);
equation
  red + blue + green = 1;
end Color;
```

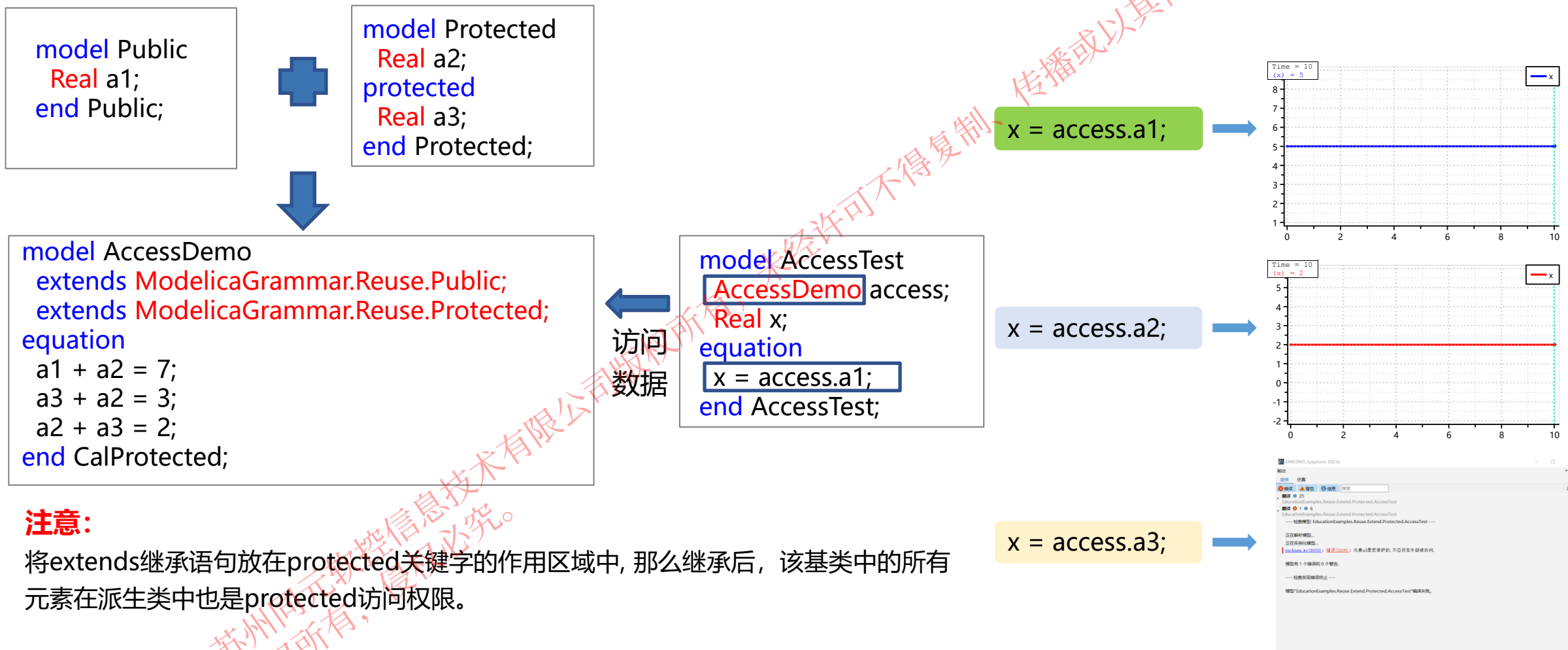


```
model Color
  parameter Real red = 0.3;
  parameter Real blue = 0.6;
  Real green;
equation
  red + blue + green = 1;
end Color;
```

一般结构：
extends + 基类路径(参数/变量变型)

1. 继承重用-注意事项

1. 继承保护



1. 继承重用-注意事项

2. 单一继承

```
record ColorData
  parameter Real red = 0.2;
  parameter Real blue = 0.6;
  Real green;
end ColorData;
```

```
model Color
  extends ColorData;
  parameter Real red = 0.2;
  parameter Real blue = 0.6;
  equation
    red + blue + green = 1;
end Color;
```

第一种声明方式

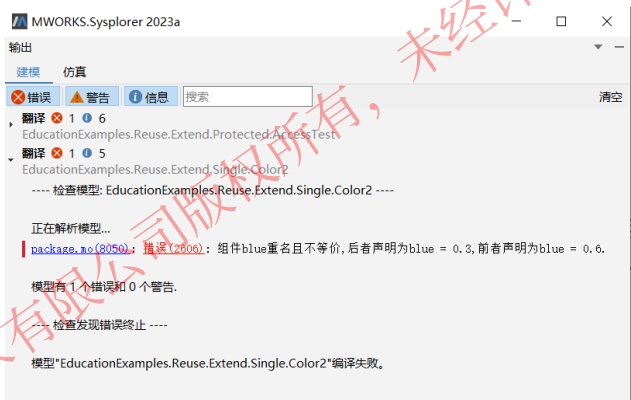
```
model Color
  extends ColorData;
  parameter Real red = 0.2;
  parameter Real blue = 0.3;
  equation
    red + blue + green = 1;
end Color;
```

第二种声明方式

声明单一继承规则：

- 继承后如果有多个相同的声明，只保留一个声明。
- 继承后如果对同一个元素有多个不同的声明，会报错。

思考：左侧的两种声明方式是否会报错？



1. 继承重用-注意事项

3.继承限制

```
record RecordA
...
end RecordA;
```



```
model ModelA
extends RecordA; //正确
end ModelA;
```

```
package PackageA
...
end PackageA;
```



```
model ModelB
extends PackageA; //错误
end ModelB;
```

	基类						
派生类	package	function	type	record	connector	block	model
package	√						
function		√					
type			√				
record				√			
connector			√	√	√		
block				√		√	
model				√		√	√

打√为派生类可继承的基类



目录

1. 继承重用

2. 实例化重用

3. 重声明重用

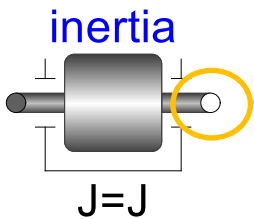
4. 本章回顾

2. 实例化重用-使用场景

- 1. 接口实例化
- 2. 模型封装
- 3. 系统仿真模型构建

场景1：接口实例化

新开发的模型中需要接口，可直接通过实例化的方式进行引用

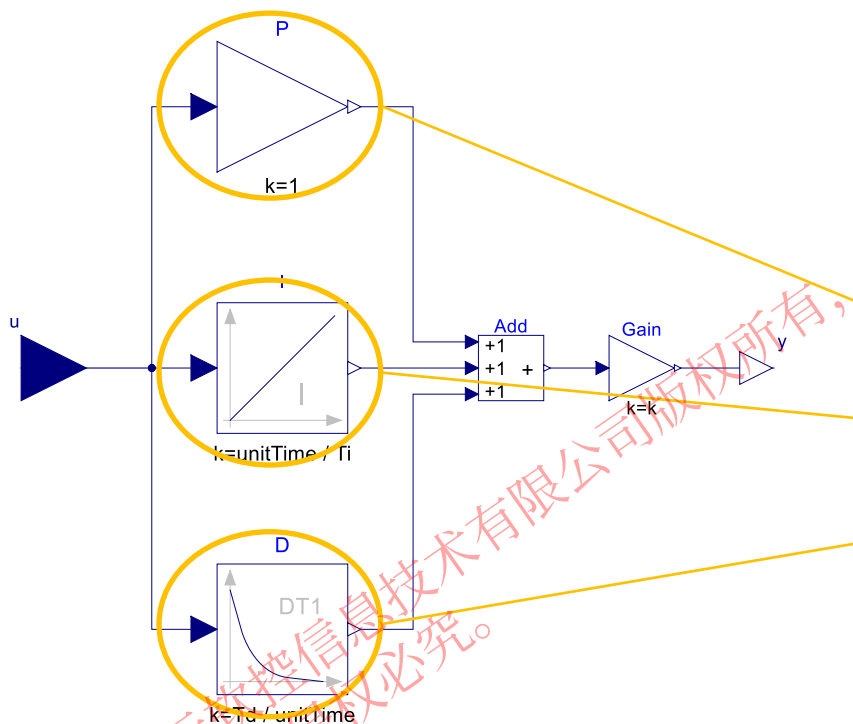


```
model Inertia "1D-rotational component with inertia"
  Rotational.Interfaces.Flange_a flange_a "Left flange of shaft"
    annotation (Placement(transformation(extent = {{-110, -10}, {-90, 10}}, ...
  Rotational.Interfaces.Flange_b flange_b "Right flange of shaft"
    annotation (Placement(transformation(extent = {{90, -10}, {110, 10}}, ...
  parameter SI.Inertia J(min = 0, start = 1) "Moment of inertia";
  parameter StateSelect stateSelect = StateSelect.default
    "Priority to use phi and w as states"
    annotation (HideResult = true, Dialog(tab = "Advanced"));
  SI.Angle phi(stateSelect = stateSelect)
    "Absolute rotation angle of component"
    annotation (Dialog(group = "Initialization", showStartAttribute = true));
  SI.AngularVelocity w(stateSelect = stateSelect)
    "Absolute angular velocity of component (= der(phi))"
    annotation (Dialog(group = "Initialization", showStartAttribute = true));
  SI.AngularAcceleration a
    "Absolute angular acceleration of component (= der(w))"
    annotation (Dialog(group = "Initialization", showStartAttribute = true));
equation
  phi = flange_a.phi;
  phi = flange_b.phi;
  w = der(phi);
  a = der(w);
  J * a = flange_a.tau + flange_b.tau;
end Inertia;
```

2. 实例化重用-使用场景

场景2：模型封装

PID控制器，由比例单元(P)、积分单元(I)和微分单元(D)组成



```
block PID "PID-controller in additive description form"
  extends Interfaces.SISO;
  parameter Real k(unit = "1") = 1 "Gain";
  parameter SIunits.Time Ti(min = Modelica.Constants.small, start = 0.5);
  parameter SIunits.Time Td(min = 0, start = 0.1);
  parameter Real Nd(min = Modelica.Constants.small) = 10;
  parameter Modelica.Blocks.Types.InitPID initType =
    Modelica.Blocks.Types.InitPID.DoNotUse_InitialIntegratorState;
  parameter Real xi_start = 0;
  parameter Real xd_start = 0;
  parameter Real y_start = 0;
  constant SI.Time unitTime = 1;

  Blocks.Math.Gain P(k = 1) "Proportional part of PID controller"
    annotation (Placement(transformation(extent = {{-60, 60}, {-20, 100}}, ...

  Blocks.Continuous.Integrator I(k = unitTime / Ti, y_start = xi_start, ...
    annotation (Placement(transformation(extent = {{-60, -20}, {-20, 20}}, ...

  Blocks.Continuous.Derivative D(k = Td / unitTime, ...
    annotation (Placement(transformation(extent = {{-60, -100}, {-20, -60}}, ...

  Blocks.Math.Gain Gain(k = k) "Gain of PID controller"
    annotation (Placement(transformation(extent = {{60, -10}, {80, 10}}, rotation = ...

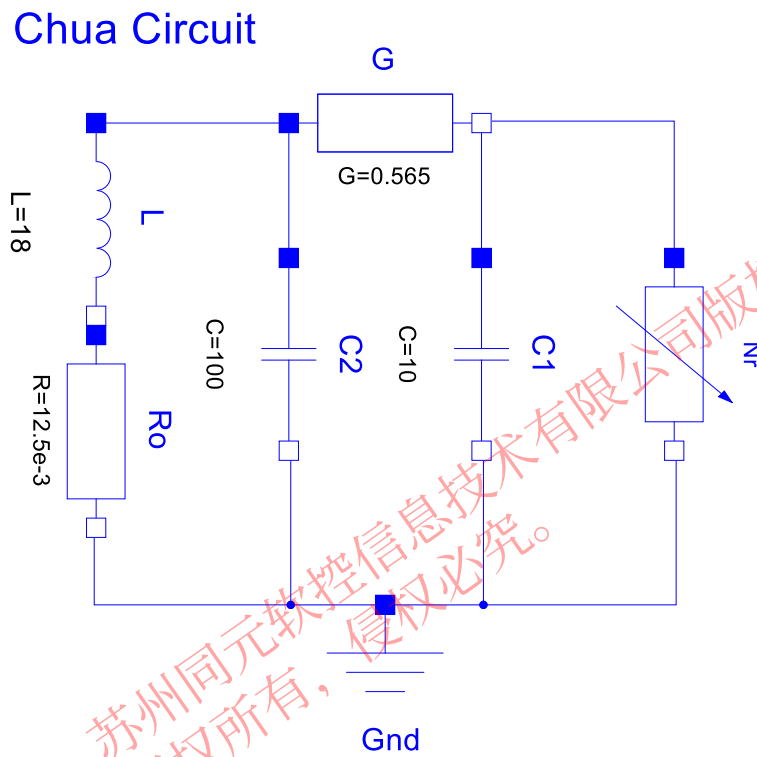
  Blocks.Math.Add3 Add annotation (Placement(transformation(extent = {{20, -10}, ...
```



2. 实例化重用-使用场景

场景3：系统仿真模型构建

蔡氏电路：一种简单的非线性电子电路设计，它可以表现出标准的混沌理论行为。



```
model ChuaCircuit "Chua's circuit, ns, V, A"
  import Modelica.Electrical.Analog.Basic;
  import Modelica.Electrical.Analog.Examples.Utilities;
  import Modelica.Icons;
  extends Icons.Example;

  Basic.Inductor L(L = 18, i(start = 0, fixed = true)) annotation (...);
  Basic.Resistor Ro(R = 12.5e-3) annotation (...);
  Basic.Conductor G(G = 0.565) annotation (...);
  Basic.Capacitor C1(C = 10, v(start = 4, fixed = true)) annotation (...);
  Basic.Capacitor C2(C = 100, v(start = 0, fixed = true)) annotation (...);
  Utilities.NonlinearResistor Nr(...) annotation (...);
  Basic.Ground Gnd annotation (...);

equation
  connect(L.n, Ro.p) annotation (Line(points = {{-75, 13}, {-75, 8}}));
  connect(C2.p, G.p) annotation (Line(...));
  connect(L.p, G.p) annotation (Line(...));
  connect(G.n, Nr.p) annotation (Line(...));
  connect(C1.p, G.n) annotation (Line(...));
  connect(Ro.n, Gnd.p) annotation (Line(...));
  connect(C2.n, Gnd.p) annotation (Line(...));
  connect(Gnd.p, C1.n) annotation (Line(...));
  connect(Gnd.p, Nr.n) annotation (Line(...));
  annotation (...);
end ChuaCircuit;
```

2. 实例化重用-一般结构

```
Interfaces.FluidInterfaces.FluidPort_a port_a(p(start = p_start));
```

- connector重用

```
Modelica.Media.Water.WaterIF97_base.ThermodynamicState state;
```

- record重用

```
Modelica.Electrical.Analog.Basic.Resistor R3(R = 1.e-3);
```

- model重用

```
Modelica.SIunits.Pressure p(displayUnit = "MPa");
```

- type重用

一般结构：

父类模型路径+实例化名（参数、变量、特殊类变型）

2. 实例化重用-一般结构

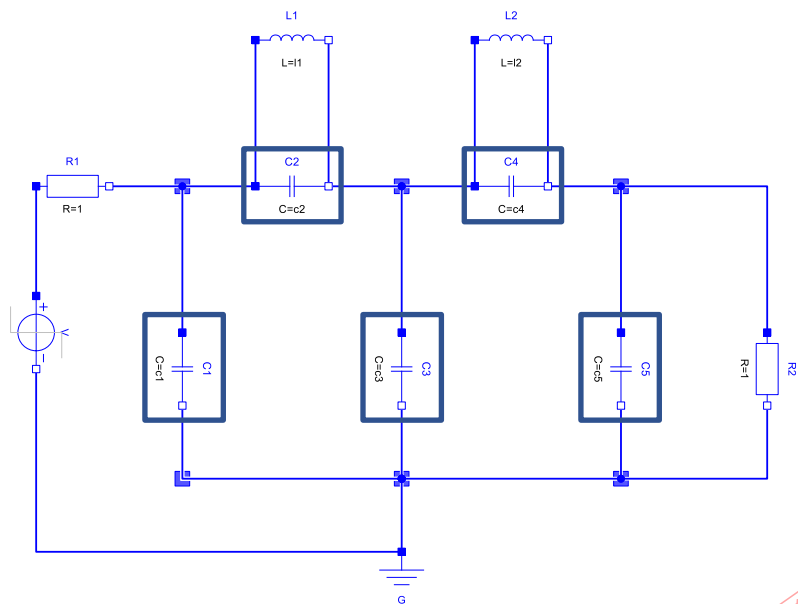
父类模型路径：待实例化的类在模型库中的路径，可通过拖拽类的方式直接生成

实例化名：符合Modelica命名规则，并能形象描述实例化类的作用

变型：对实例化后的类参数、变量初始值、变量、显示单位、特殊类（replaceable/redeclare 单独讲解）进行变型

2. 实例化重用-使用方式

1. 参数变型



```
model CauerLowPassAnalog
  extends Modelica.Icons.Example;
  Modelica.Electrical.Analog.Basic.Capacitor C1(C = c1, v(start = 0, fixed = true))
    annotation (...);
  Modelica.Electrical.Analog.Basic.Capacitor C2(C = c2)
    annotation (...);
  Modelica.Electrical.Analog.Basic.Capacitor C3(C = c3, v(start = 0, fixed = true))
    annotation (...);
  Modelica.Electrical.Analog.Basic.Capacitor C4(C = c4)
    annotation (...);
  Modelica.Electrical.Analog.Basic.Capacitor C5(C = c5, v(start = 0, fixed = true))
    annotation (...);
  ...
end CauerLowPassAnalog;
```

实例化C1、C2、C3、C4、C5，通过变型设置电容值，这样就不必因为电容参数的不同而建立多个电容模型。

注意：

目前遇到的大多数示例中，被变型的组件属性都是参数。尽管Modelica规范中没有明确禁止对变量、常量等进行变型，但鉴于Modelica参数的设计意图，**不推荐对变量、常量等进行变型。**

组件参数			
常规			
参数			
I1	1.304	H	filter coefficient I1
I2	0.8586	H	filter coefficient I2
c1	1.072	F	filter coefficient c1
c2	$1 / (1.704992 \times 2 * I1)$	F	filter coefficient c2
c3	1.682	F	filter coefficient c3
c4	$1 / (1.179945 \times 2 * I2)$	F	filter coefficient c4
c5	0.7262	F	filter coefficient c5

2. 实例化重用-使用方式

2.数组变型

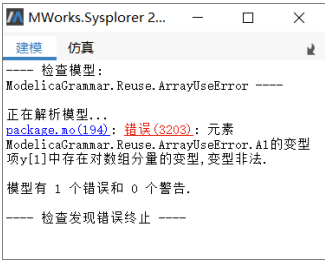
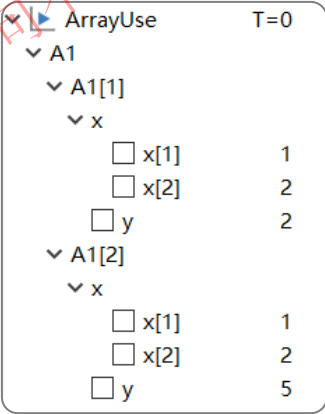
```
model Array
  parameter Real x[2];
  parameter Real y;
end Array;
```

```
model ArrayUse
  Array A1[2](x = [1, 2; 1, 2], y = {2, 5});
end ArrayUse;
```



```
model ArrayUse
  Array A1[2](each x = {1, 2}, y = {2, 5});
end ArrayUse;
```

```
model ArrayUseError
  Array A1[2](x = [1, 2; 1, 2], y[1] = 2);
end ArrayUse;
```



注意:

- 使用“each”对所有数组元素中的某个属性赋相同的值。
- 必须对整个数组进行变型, 不可以对单个数组元素进行变型。

2. 实例化重用-使用方式

3.单一变型

在一次变型中，**不能**对一个元素的同一个属性进行两次赋值修改

```
model M1
  Real x[3];
end M1;
```

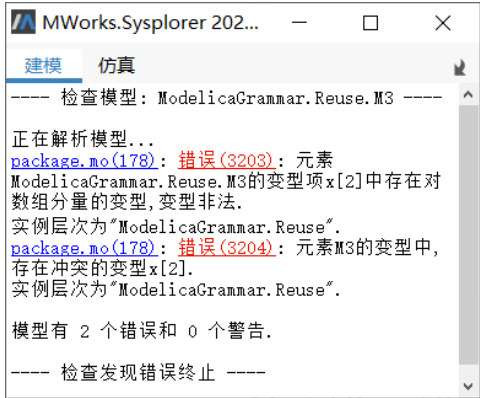
```
ModelicaGrammar.Reuse.M1 M2(x = ones(3));
```

正确

名字	值
✓ M2	T=1
x	
x[1]	1
x[2]	1
x[3]	1

```
ModelicaGrammar.Reuse.M1 M3(x = ones(3),x[2]=2);
```

错误



2. 实例化重用-注意事项

变型合并

```
model C1
  parameter Real a;
end C1;

model C2
  parameter Real b, c;
end C2;

model C3
  final parameter Real x1 = 1;
  parameter Real x2 = 2;
  C1 x3[2](each a = 33);
  C2 x4(b = 4);
  C1 x5(a = 5);
  extends C1;
  extends C2(b = 6, c = 77);
end C3;

model C4
  extends C3(x2 = 22, x4(c = 44), x5 = x3, a = 55, b = 66);
end C4;
```

变量	结果
x1	1
x2	22
x3[1].a	33
x3[2].a	33
x4.b	4
x4.c	44
x5.a	x3.a
a	55
b	66
c	77

C4变量赋值结果

- 注意：**
- 每个元素只能赋值一次；
 - final元素不能进行赋值；
 - each表示数组中的每个变量均赋相同的值；
 - 变形按照外层覆盖内层的原则进行合并；

目录

1. 继承重用

2. 实例化重用

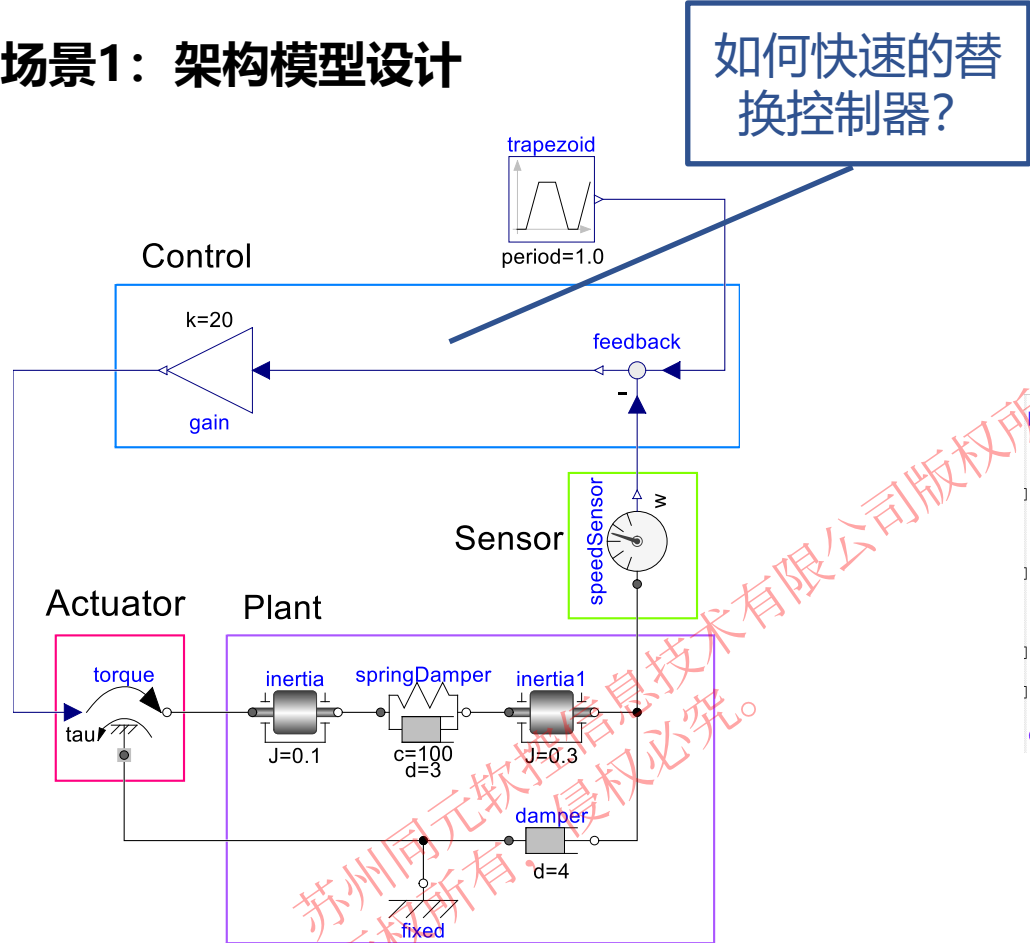
3. 重声明重用

4. 本章回顾

3. 重声明重用-使用场景

- 1. 架构模型设计
- 2. 介质数据库

场景1：架构模型设计



可替换重申明（ replaceable/ redeclare ）是其中一种快速替换模型的方法。

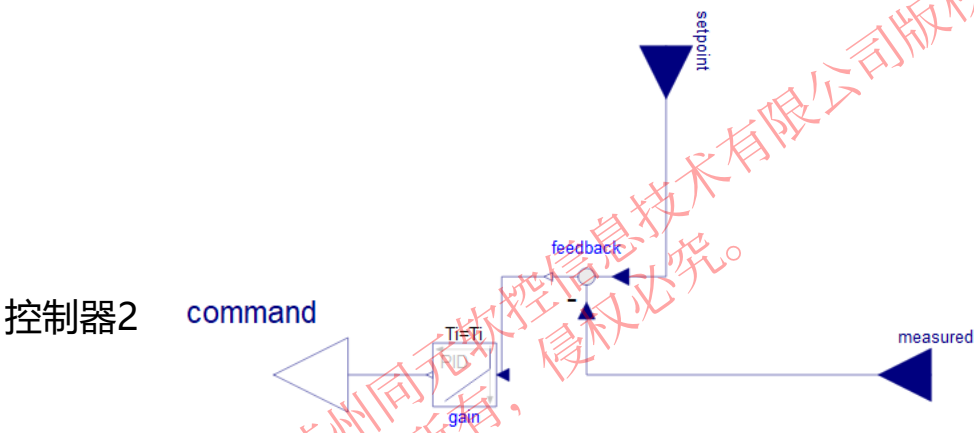
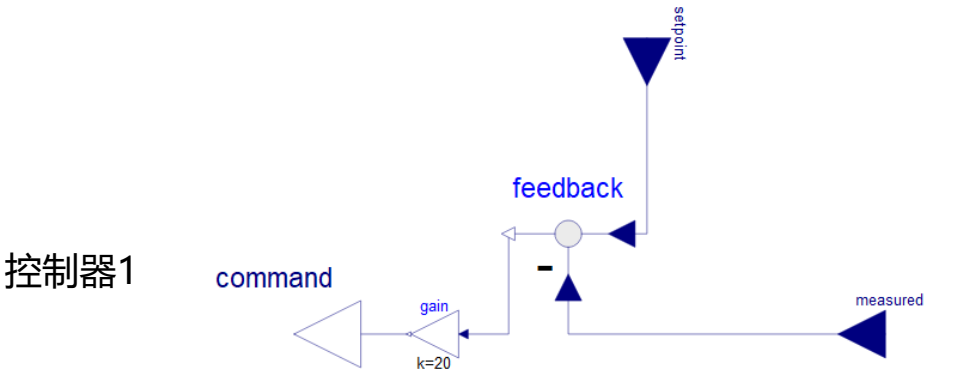
- ①. 观察该系统的控制器，有三个接口，两个输入，一个输出，那么我们需要替换的控制器具有相同的接口数量和接口类型。
- ②. 设计一个架构模型，它只包含了接口，两个输入、一个输出

```
model PartialController "Interface for controller subsystem"
  Modelica.Blocks.Interfaces.RealInput setpoint "Desired system response"
  annotation (Placement(transformation(...)))
  Modelica.Blocks.Interfaces.RealInput measured "Actual system response"
  annotation (Placement(transformation(...)))
  Modelica.Blocks.Interfaces.RealOutput command "Command to send to actuator"
  annotation (Placement(transformation(...)))
  annotation (Icon(coordinateSystem(extent = {{-100.0, -100.0}, {100.0, 100.0}}), ...))
end PartialController;
```



3. 重声明重用-使用场景

③. 继承架构模型，开发不同的控制器模型

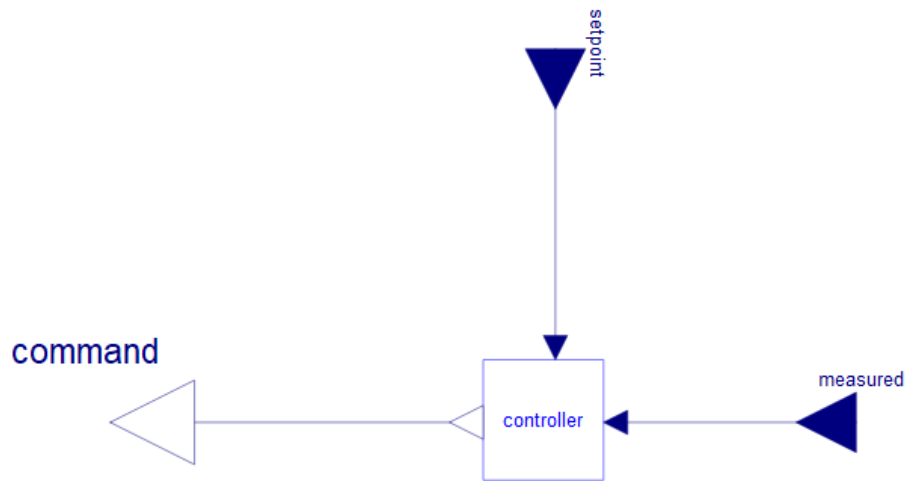


```
model Controller1
  extends ReplaceableDemo.Interfaces.PartialController;
  Modelica.Blocks.Math.Feedback feedback annotation (...);
  Modelica.Blocks.Math.Gain gain(k = 20) annotation (...);
equation
  connect(gain.u, feedback.y)
    annotation (Line(origin = {-34.0, 21.0}, ...));
  connect(feedback.u2, measured)
    annotation (Line(origin = {42.0, 17.0}, ...));
  connect(feedback.u1, setpoint)
    annotation (Line(origin = {-4.0, 81.0}, ...));
  connect(gain.y, command)
    annotation (Line(origin = {-88.0, 0.0}, ...));
end Controller1;

model Controller2
  extends ReplaceableDemo.Interfaces.PartialController;
  Modelica.Blocks.Math.Feedback feedback annotation (...);
  Modelica.Blocks.Continuous.PID gain(k = 20) annotation (...);
equation
  connect(gain.u, feedback.y)
    annotation (Line(origin = {-34.0, 21.0}, ...));
  connect(feedback.u2, measured)
    annotation (Line(origin = {42.0, 17.0}, ...));
  connect(feedback.u1, setpoint)
    annotation (Line(origin = {-4.0, 81.0}, ...));
  connect(gain.y, command)
    annotation (Line(origin = {-88.0, 0.0}, ...));
end Controller2;
```

3. 重声明重用-使用场景

④. 封装一个控制器模型，用于系统模型构建

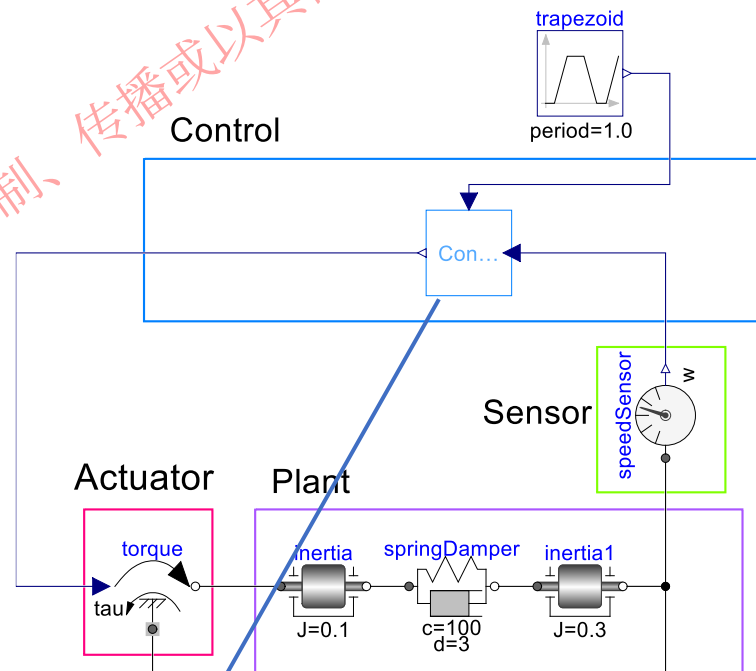


```

model Controller;
    extends ReplaceableDemo.Interfaces.PartialController;
    replaceable ReplaceableDemo.Interfaces.PartialController controller
        annotation (choicesAllMatching = true, ...)
equation
    connect(controller.setpoint, setpoint)
        annotation (Line(origin = {0.0, 66.0}, ...))
    connect(controller.measured, measured)
        annotation (Line(origin = {55.0, 0.0}, ...))
    connect(controller.command, command)
        annotation (Line(origin = {-60.0, 0.0}, ...))
end Controller;

```

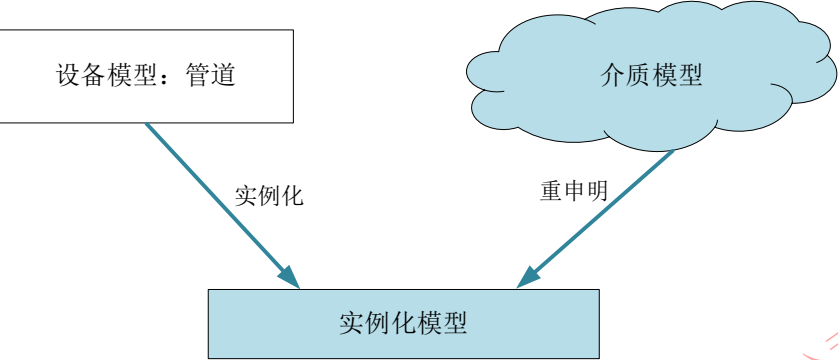
⑤. 采用封装后的控制器模型替代原系统模型中的控制组件



3. 重声明重用-使用场景

场景2：介质数据库

设想一个场景，用Modelica语言开发了一个水管模型，如果想把水管中流动的水改变为油液，用于油路系统中，该如何处理？



采用可替换重申明（ replaceable/ redeclare ）的方法，将设备模型与介质模型分离，开发设备模型时调用介质模型模板，实例化设备模型时，将具体介质与设备关联，得到带有介质的实例化设备模型。

①. 介质模板

```
partial package PartialMedium
  replaceable partial function setState_pTX
    input AbsolutePressure p "Pressure";
    input Temperature T "Temperature";
    input MassFraction X[:] = reference_X "Mass fractions";
    output ThermodynamicState state "Thermodynamic state record";
  end setState_pTX;

  replaceable partial function dynamicViscosity "Return dynamic viscosity"
    input ThermodynamicState state "Thermodynamic state record";
    output DynamicViscosity eta "Dynamic viscosity";
  end dynamicViscosity;

  .....
end PartialMedium;
```

②. 水介质物性计算

```
partial package WaterIF97_base "Water"
  extends Interfaces.PartialTwoPhaseMedium( ... )
  redeclare function extends setState_pTX
    "Return thermodynamic state of water as function of p and T"
  algorithm
    state := ThermodynamicState( ... )
  end setState_pTX;

  redeclare function extends dynamicViscosity "Dynamic viscosity of water"
  algorithm
    eta := IF97_Uutilities.dynamicViscosity( ... )
  end dynamicViscosity;

  .....
end WaterIF97_base;
```

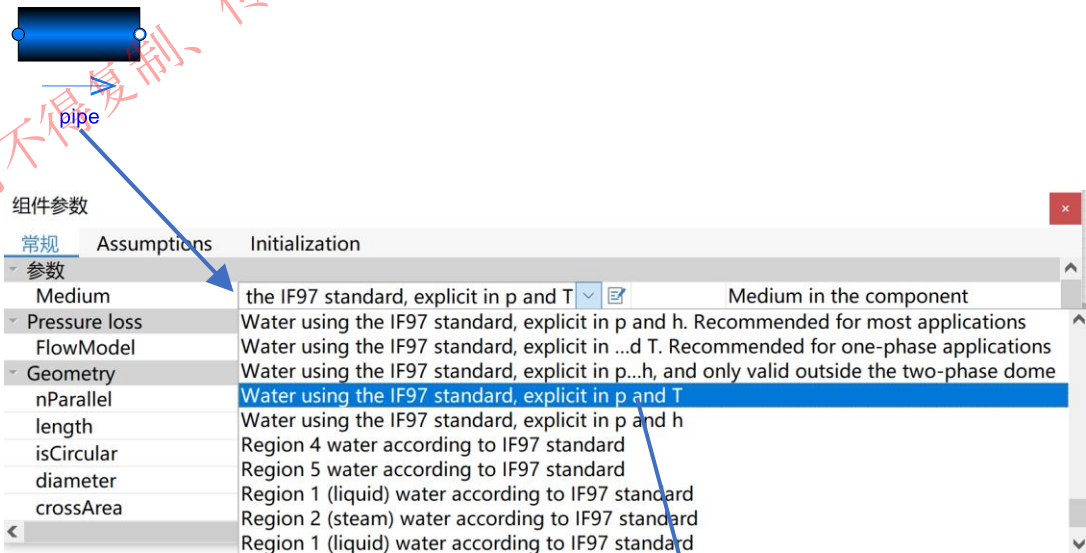


3. 重声明重用-使用场景

③. 设备模型开发时与介质模板关联，并调用介质模板函数进行代码编写

```
model StaticPipe "Basic pipe flow model without storage of mass or energy"
  replaceable package Medium = Modelica.Media.Interfaces.PartialMedium
    "Medium in the component"
  annotation (choicesAllMatching = true);
  Medium.ThermodynamicState state = Medium.setState_pTX(port_a.p,T);
  Modelica.SIunits.DynamicViscosity mu = Medium.dynamicViscosity(state);
  .....
equation
  // Mass balance
  port_a.m_flow = flowModel.m_flows[1];
  0 = port_a.m_flow + port_b.m_flow;
  .....
  annotation (defaultComponentName = "pipe", ...);
end StaticPipe;
```

④. 实例化管道模型，并替换介质模板，得到带有介质的管道实例化模型



```
Modelica.Fluid.Pipes.StaticPipe pipe(
  redeclare package Medium = Modelica.Media.Water.WaterIF97_pT
  annotation (Placement(transformation(...
```



3. 重声明重用-使用方法

重声明重用，在语法中通过`replaceable/ redeclare`关键字实现，根据Modelica标准语法规义的规定可以对所有的类、参数、变量进行重声明，但是在实际的使用中以`model`、`package`、`function`这三种类的重声明为主，以实现通用化模型的开发。

关键字: `replaceable`: 表示类型或组件可以被替换

`redeclare`: 表示替换类型或组件

3. 重声明重用-使用方法

1.model重申明

基类模型

```
model PartialGeometry
  Modelica.SIunits.Area A "横截面积";
  Modelica.SIunits.Length l "周长";
end PartialGeometry;
```

圆形

```
model cycle "圆形"
  extends PartialGeometry;
  parameter Modelica.SIunits.Diameter d = 0.01;
equation
  A = Modelica.Constants.pi * d ^ 2 / 4;
  l = Modelica.Constants.pi * d;
end cycle;
```

三角形

```
model triangle "三角形"
  extends PartialGeometry;
  parameter Modelica.SIunits.Length a = 0.01;
equation
  A = 0.5 * a * a * sqrt(3) / 2;
  l = 3 * a;
end triangle;
```

正方形

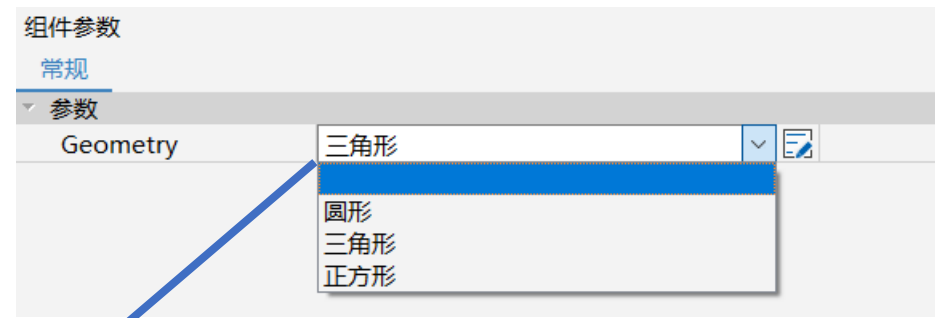
```
model square "正方形"
  extends PartialGeometry;
  parameter Modelica.SIunits.Length a = 0.01;
equation
  A = a ^ 2;
  l = 4 * a;
end square;
```



```
model Test_Geometry
  replaceable model Geometry = cycle
    constrainedby PartialGeometry
    annotation (choicesAllMatching = true);
  Geometry geometry;
  Modelica.SIunits.Area A "横截面积";
  Modelica.SIunits.Length l "周长";
equation
  A = geometry.A;
  l = geometry.l;
end Test_Geometry;
```



实例化后使用模型



```
Test.Test_Geometry test_Geometry(
  redeclare model Geometry = Test.triangle)
```



3. 重声明重用-使用方法

1.model重申明

典型应用场景：模型的几何结构、不同换热计算方法、不同流阻计算方法等

典型应用案例：标准库中DynamicPipe模型换热模式和流阻计算公式的切换，模型路径：

Modelica.Fluid.Pipes.DynamicPipe

```
model DynamicPipe "Dynamic pipe model with storage of mass and energy"
import Modelica.Fluid.Types.ModelStructure;
extends Modelica.Fluid.Pipes.BaseClasses.PartialStraightPipe(...
extends BaseClasses.PartialTwoPortFlow(...
parameter Boolean use_HeatTransfer = false "= true to use the HeatTransfer model"
annotation (Dialog(tab = "Assumptions", group = "Heat transfer"));
replaceable model HeatTransfer = Modelica.Fluid.Pipes.BaseClasses.HeatTransfer.IdealFlowHeatTransfer
constrainedby Modelica.Fluid.Pipes.BaseClasses.HeatTransfer.PartialFlowHeatTransfer "Wall heat transfer"
annotation (Dialog(tab = "Assumptions", group = "Heat transfer", enable = use_HeatTransfer), choicesAllMatching = true);
HeatTransfer heatTransfer(
redeclare final package Medium = Medium,
final n = n,
final nParallel = nParallel,
final surfaceAreas = perimeter * lengths,
final lengths = lengths,
final dimensions = dimensions,
final roughnesses = roughnesses,
final states = mediums.state,
final vs = vs,
final use_k = use_HeatTransfer) "Heat transfer model"
annotation (Placement(transformation(extent = {{-45, 20}, {-23, 42}}, rotation = 0)));
```

组件参数			
常规	Assumptions	Initialization	Advanced
参数			
allowFlowReversal	system.allowFlowReversal		= true to allow flow reversal, false restricts to design direction (port_a -> port_b)
Dynamics			
energyDynamics	system.energyDynamics		Formulation of energy balances
massDynamics	system.massDynamics		Formulation of mass balances
momentumDynamics	system.momentumDynamics		Formulation of momentum balances
Heat transfer			
HeatTransfer	transfer without thermal resistance		Wall heat transfer
use_HeatTransfer	IdealHeatTransfer: Ideal heat transfer without thermal resistance ConstantHeatTransfer: Constant heat transfer coefficient LocalPipeFlowHeatTransfer: Laminar and turbulent forced convection in pipes, local coefficients		



3. 重声明重用-使用方法

2.package和function重声明

```
package PartialSolidMedium "固体介质基类"
replaceable partial function Cp_T "返回定压比热"
  input SI.Temperature T;
  output SI.SpecificHeatCapacity Cp "定压比热";
end Cp_T;

replaceable partial function thermalConductivity_T "计算导热系数"
  input SI.Temperature T;
  output SI.ThermalConductivity lambda "导热系数";
end thermalConductivity_T;

replaceable partial function rho_T "返回密度"
  input SI.Temperature T;
  output SI.Density rho "密度";
end rho_T;
end PartialSolidMedium;
```

基类模型

```
package Graphite "石墨,没有受过辐射"
extends PartialSolidMedium;
constant Real c1(unit = "J/(kg.K)") = -143.9883;
constant Real c2(unit = "J/(kg.K2)") = 3.6677;
constant Real c3(unit = "J/(kg.K3)") = -0.0022;
constant Real c4(unit = "J/(kg.K4)") = 4.6251e-7;
redeclare function extends rho_T "计算密度"
algorithm
  rho := 1776.66;
end rho_T;
redeclare function extends Cp_T "计算比热容"
algorithm
  Cp := c1 + c2 * T + c3 * T ^ 2 + c4 * T ^ 3;
end Cp_T;
redeclare function extends thermalConductivity_T "计算导热系数"
algorithm
  lambda := 169.245 - 1.24890e-1 * T + 3.28248e-5 * T ^ 2;
end thermalConductivity_T;
end Graphite;
```

模型1

```
package B4C "碳化硼"
extends PartialSolidMedium;
redeclare function extends rho_T "计算密度"
algorithm
  rho := 1800;
end rho_T;
redeclare function extends Cp_T "计算比热容"
algorithm
  Cp := 713;
end Cp_T;
redeclare function extends thermalConductivity_T "计算导热系数"
algorithm
  lambda := (-1.873e-10) * T ^ 4 + (5.792e-07) * T ^ 3 + (-0.0005904) * T ^ 2 + 0.1406 * T + 119.7;
end thermalConductivity_T;
end B4C;
```

模型2

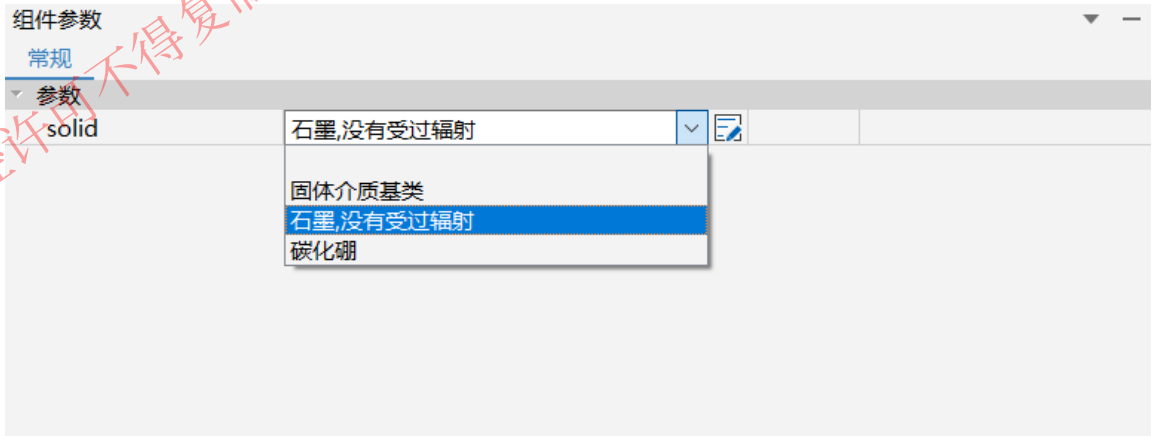
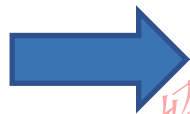


3. 重声明重用-使用方法

2.package和function重声明

```
model Test_packagefunction
  replaceable package solid = Graphite
    constrainedby PartialSolidMedium
    annotation (choicesAllMatching = true);
  Modelica.SIunits.Temperature T = 300 "温度";
  Modelica.SIunits.Density rho;
  Modelica.SIunits.SpecificHeatCapacity Cp;
  Modelica.SIunits.ThermalConductivity lamda;
equation
  rho = solid.rho_T(T);
  Cp = solid.Cp_T(T);
  lamda = solid.thermalConductivity_T(T);
end Test_packagefunction;
```

应用模型



3. 重声明重用-使用方法

2.package和function重声明

典型应用场景：流体系统中，设备模型与介质模型的分离，通过基于package和function的重声明，实现设备模型中不同类型的介质选择；

典型应用案例：Modelica标准库中Media介质库的开发和调用方式；

3. 重声明重用-注意事项

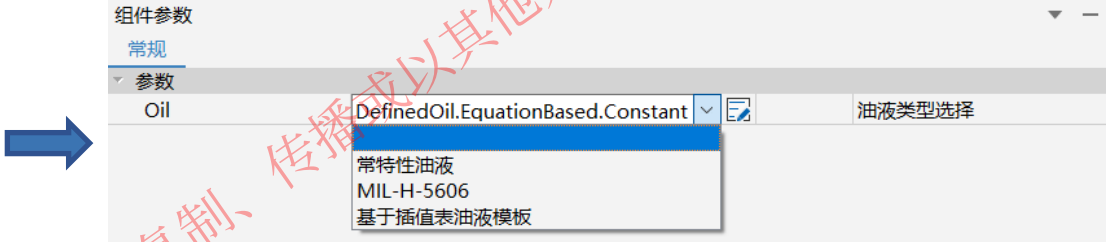
1.重声明约束

```
model Oil "液压油"  
  replaceable model Oil = PreDefinedOil.EquationBased.Constant  
    constrainedby OilMedia.Interfaces.Base "油液类型选择"  
    annotation (choicesAllMatching = true);  
  annotation (...);  
end Oil;
```

```
model Constant "常特性油液"  
  extends OilMedia.Interfaces.Base(...);  
  annotation (...);  
end Constant;
```

```
model MIL_H_5606 "MIL-H-5606牌号油液"  
  extends OilMedia.Interfaces.Base(...);  
  annotation (...);  
end MIL_H_5606 ;
```

```
model Tabular "基于插值表油液模板"  
  extends OilMedia.Interfaces.Base(...);  
  annotation (...);  
end Tabular;
```



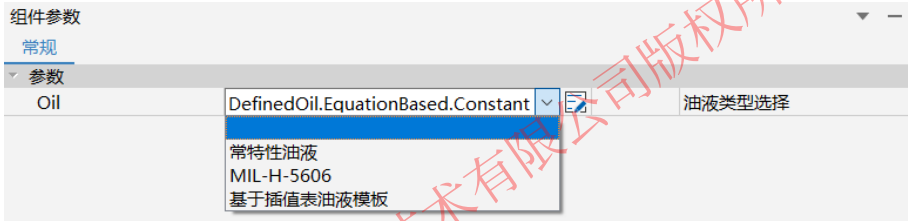
说明:

- “constrainedby 基类路径” 用来筛选所有继承此基类的模型
- “choicesAllMatching = true” 表示所有符合筛选条件的模型均可在参数框中进行选取重声明

3. 重声明重用-注意事项

2.重声明注解

```
model Oil "液压油"  
  replaceable model Oil = PreDefined.EquationBased.Constant  
  constrainedby PreDefinedOil.EquationBased.Constant "油液类型选择"  
  annotation (choices(  
    choice(redeclare model Oil = OilMedia.PreDefined.EquationBased.Constant "常特性油液"),  
    choice(redeclare model Oil = OilMedia.PreDefined.TabularBased.MIL_H_5606 "MIL-H-5606"),  
    choice(redeclare model Oil = OilMedia.Templates.Tabular "基于插值表油液模板")));  
  annotation (...);  
end Oil;
```



说明:

- “constrainby 组件路径” 表示默认选择的模型
- “choices” 作用为增加下拉框选项并赋值，详见《注解》章节

目录

1. 继承重用

2. 实例化重用

3. 重声明重用

4. 本章回顾

4. 本章回顾

● 继承重用

使用场景：

- 继承connector，设计不同的图标和连接线；
- 继承partial类，完善模型行为方程；

一般结构：

- extends + 基类路径(参数/变量变型)

注意事项：

- 继承保护
- 继承限制
- 单一继承

● 重申明重用

使用场景：

- 架构模型设计；
- 介质数据库；

使用方法：

- model重申明
- package和function重声明

注意事项：

- 重申明约束
- 重申明注解

● 实例化重用

使用场景：

- 接口实例化；
- 模型封装；
- 系统仿真模型构建

一般结构：

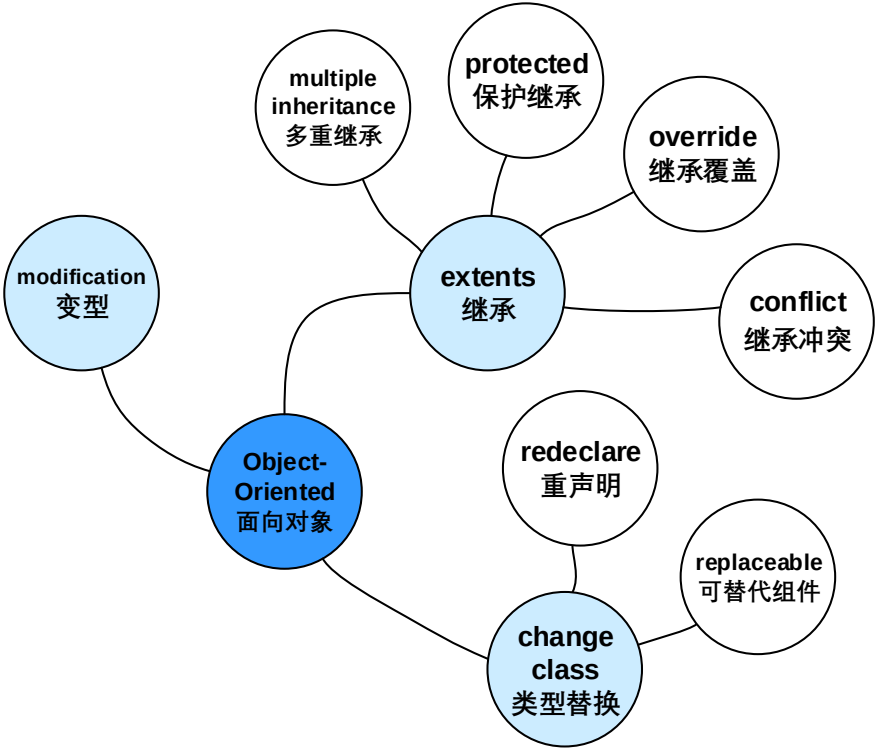
- 父类模型路径+ 实例化名（参数、变量、特殊类变型）

使用方法：

- 参数变型
- 数组变型

注意事项：

- 单一变型
- 变型合并



4. 本章回顾

课堂回顾

1. 继承一个模型需要用到的关键词是 () 。
A. partial B. extends C. public D. record
2. 下列类中，不能继承record的是 () 。
A. package B. record C. connector D. model
3. 对数组变型的时候，用到的关键词是 () 。
A. Real B. class C. each D. change
4. () 关键词定义的元素禁止变型。
A. public B. type C. final D. partial
5. 下面是重声明的关键词的是 () 。
A. replaceable B. pre C. change D. edge
6. 用于替换类型或组件的关键词是 () 。
A. change B. record C. cross D. redeclare
7. 表明组件可被替换的关键词是 () 。
A. replaceable B. pre C. change D. edge
8. 继承保护的关键词是 () 。
A. partial B. public C. protected D. change
9. V型继承属于 () 继承。
10. 可以继承package的类是 () 。
11. 重声明的两个关键词分别是 () 和 () 。
12. 外层变型可以覆盖 () 变型。
13. 在声明一个对象的同时修改类的属性属于 () 。

4. 本章回顾

课后作业

使用可替换重申明（ replaceable/ redeclare ） ，
开发一个管道规格数据模型库；开发一个管道质量
计算模型，通过选择不同的管道规格，并在参
数面板中设置管道长度，计算得到该管道的质量。

规格		外径 (D)mm	壁厚 (t)mm
公称内径	英寸		
DN6		10.2	2.0
DN8		13.5	2.5
DN10		17.2	2.5
DN15	1/2	21.3	2.8
DN20	3/4	26.9	2.8
DN25	1	33.7	3.2
DN32	1.25	42.4	3.5
DN40	1.5	48.3	3.5
DN50	2	60.3	3.8
DN65	2.5	76.1	4.0
DN80	3	88.9	4.0
DN100	4	114.3	4.0
DN125	5	139.7	4.0
DN150	6	168.3	4.5

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

谢谢！